

Algoritmos Recursivos Parte 2

Algoritmos e Estrutura de Dados 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Escrevendo Programas Recursivos

- Na última aula vimos como transformar uma **definição** ou um **algoritmo recursivo** em um programa em C
- O desenvolvimento de uma solução recursiva em C para a especificação de um problema cujo algoritmo não é fornecido é uma tarefa muito mais difícil
 - As definições e os algoritmos originais precisam ser desenvolvidos
- Em termos gerais, ao enfrentar a tarefa de escrever um programa para resolver um problema recursivo, não há por que procurar por uma solução recursiva
- Entretanto, alguns problemas podem ser resolvidos em termos lógicos e com mais elegância por meio da recursividade
- Desenvolveremos uma técnica para encontrar soluções recursivas

Examinemos Função Fatorial

- Podemos identificar um **caso “trivial”** para o qual uma solução não-recursiva é imediatamente obtida
 - Este é o caso **0!**
- O próximo passo é encontrar um método para solucionar um **caso “complexo”** em termos de um caso “mais simples”
 - Ocasionalmente resulta num caso trivial
 - $4! = 4 * 3! = 4 * 3 * 2! = 4 * 3 * 2 * 1! = 4 * 3 * 2 * 1 * 0! = 4 * 3 * 2 * 1 = 24$

Fundamentos Rotina Recursiva

- Esses são os ingredientes fundamentais de uma rotina recursiva
- Conseguir definir um **caso “complexo”** em termos de um **“caso mais simples”**
- Ter um **caso “trivial”** (não-recursivo) diretamente solucionável

Solução Recursiva Multiplicação

1. O **caso trivial** é $b = 1$, que é definido como a
2. Em geral, o **caso complexo** $a \cdot b$ pode ser definido em termos de **caso mais simples** $a \cdot (b - 1)$ pela definição $a \cdot b = a \cdot (b - 1) + a$

Solução Recursiva Fibonacci

1. No caso da função de Fibonacci, dois **casos triviais** foram definidos $fib(0) = 0$ e $fib(1) = 1$
2. Um **caso complexo**, $fib(n)$, é, portanto, reduzido a dois casos mais simples: $fib(n) = fib(n - 2) + fib(n - 1)$

Obs.: Por causa da definição de $fib(n)$ como $fib(n - 2) + fib(n - 1)$, são necessários dois casos triviais diretamente definidos

Solução Recursiva Busca Binária

1. O **caso trivial** é aquele no qual não existem elementos a pesquisar ou o elemento sendo procurado encontra-se no meio do vetor
 - a. Se $low > high$, -1 é retornado
 - b. Se $x = a[mid]$, mid será retornado como resposta
2. No **caso mais complexo** de $high - low + 1$ elementos a pesquisar, a ocorrência da busca é restrita a uma das sub-regiões:
 - a. metade inferior do vetor, de low até $mid - 1$
 - b. metade superior do vetor, de $mid + 1$ até $high$

O Problema das Torres de Hanoi

- Até agora, examinamos as definições recursivas e como elas se enquadram no padrão que estabelecemos
- Examinemos agora um problema que não é especificado em termos de recursividade
 - O problema das **torres de Hanoi**
 - Vejamos como podemos usar técnicas recursivas para produzir uma solução lógico e elegante

Torres de Hanói

O Problema das Torres de Hanoi

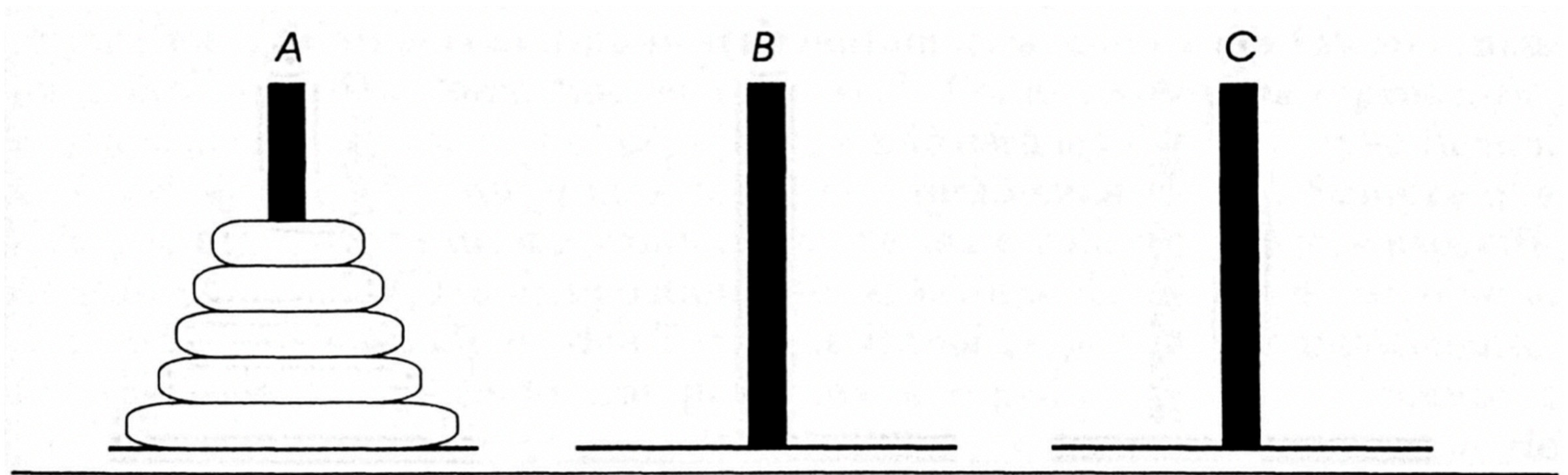
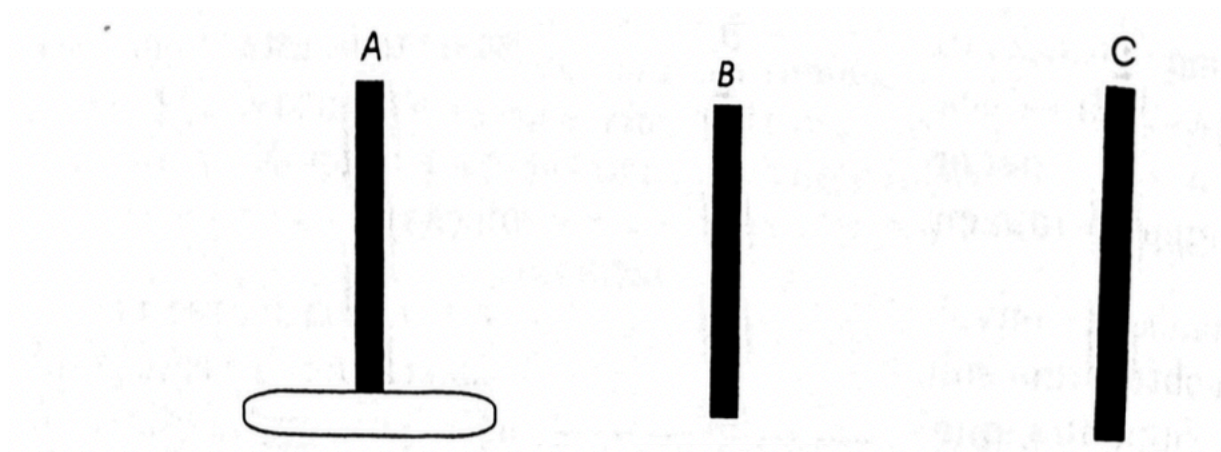


Figura 3.3.1 A configuração inicial das Torres de Hanoi.

<https://www.mathsisfun.com/games/towerofhanoi.html>

Solução Torres de Hanoi: Caso Trivial

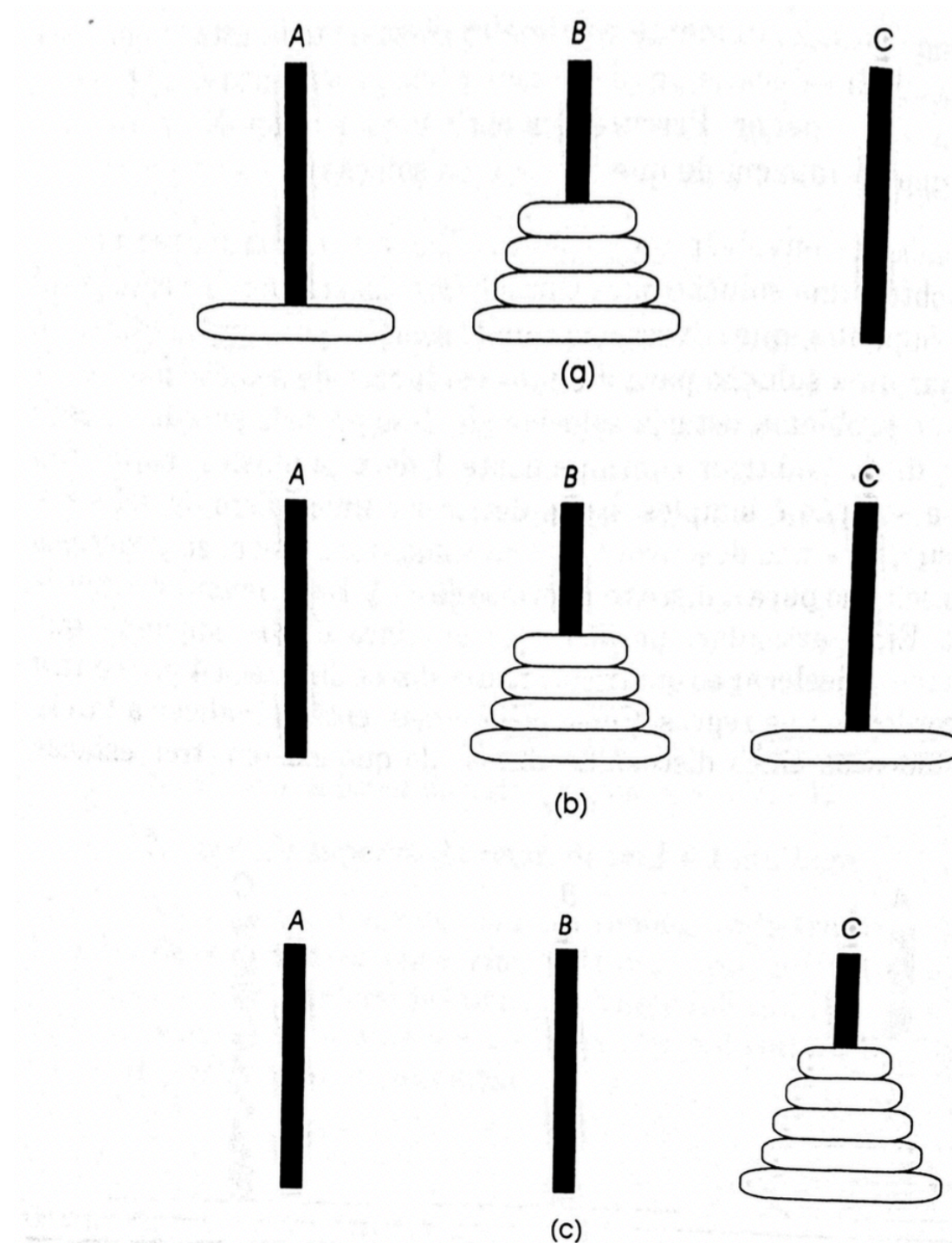
- No **caso trivial** de um disco a solução é simples: basta deslocar o único disco da estaca A para a C



Solução Torres de Hanoi: Caso Complexo

- Se pudermos declarar uma solução para n discos em termos de $n - 1$, teríamos desenvolvido uma solução recursiva ...

Solução Recursiva Torres de Hanoi



Solução Recursiva Torres de Hanoi (2/3)

- Para mover n discos de A para C, usando B como auxiliar:
 1. Se $n = 1$, desloque o único disco de A para C e pare.
 2. Desloque os $n - 1$ primeiros discos de A para B, usando C como auxiliar.
 3. Desloque o último disco de A para C.
 4. Mova os $n - 1$ discos de B para C, usando A como auxiliar.

Solução Recursiva Torres de Hanoi (3/3)

- Elaboração das entradas e saídas é importante p/ solução:
 - Usuário não verá o método elegante que você incorporou em seu programa, mas “dará duro” para decifrar a saída ou para adaptar os dados de entrada
 - Pequena mudança no formato de entrada ou da saída pode tornar o programa muito mais fácil de elaborar

Algoritmo Torres de Hanoi

```
1. #include <stdio.h>
2.
3. void towers (int n, char frompeg, char topeg, char auxpeg) {
4.     // se existe um soh disco, faca o movimento e retorne
5.     if (n == 1) {
6.         printf("mover disco 1 da estaca %c p/ a estaca %c\n", frompeg,
7. topeg);
8.     }
9.     else {
10.        // mover os primeiros n - 1 discos de A p/ B
11.        towers(n - 1, frompeg, auxpeg, topeg);
12.        printf("mover disco %d da estaca %c p/ a estaca %c\n", n, frompeg,
13. topeg);
14.        // mover n - 1 discos de B p/ C usando A como auxiliar
15.        towers(n - 1, auxpeg, topeg, frompeg);
16.    }
17.}
18.
```


Torres de Hanoi

- Existem três torres
- 64 discos de ouro, com tamanhos decrescentes, colocados na primeira torre
- Os discos devem ser movidos dentro de uma semana. Suponha que um disco possa ser movido em 1 segundo. Isso é possível?
- Para criar um algoritmo para resolver este problema, é conveniente generalizar o problema para o problema "N-disco", onde no nosso caso $n = 64$.

Movimentos Necessários

- Movimentos necessários para resolver, em função de **n**, o número de discos a serem movidos.

n	# de mov necessários
1	1
2	3
3	7
4	15
5	31
...	
i	$2^i - 1$
64	$2^{64} - 1$ (número grande)

Conversão da Forma Prefixa para a
Posfixa usando Recursividade

Notação Prefixa p/ Posfixa

Examinemos outro problema para o qual a solução recursiva é mais direta e adequada

infixo	prefixo	posfixo
$A+B$	$+AB$	$AB+$
$A + B * C$	$+A * BC$	$ABC * +$
$A * (B + C)$	$*A + BC$	$ABC + *$
$A * B + C$	$+ * ABC$	$AB * C +$
$A + B * C + D - E * F$	$- + + A * BCD * EF$	$ABC * + D + EF * -$
$(A + B) * (C + D) - E) * F$	$* * + AB - + CDEF$	$AB + CD + E - * F *$

Recursividade

- O método mais adequado para definir as formas posfixa e prefixa é pela recursividade
- Presumindo a inexistência de constantes e usando somente letras isoladas como variáveis, uma expressão prefixa é uma única letra ou um operador seguido por duas expressões prefixas
- De modo semelhante
 - Uma expressão posfixa pode ser definida como uma única letra, ou como um operador precedido por duas expressões posfixas

Algoritmo

Podemos resumir nosso algoritmo assim:

1. Se a string prefixa for uma única variável, ele será seu próprio equivalente posfixo
2. Considere *op* como o primeiro operador da string prefixa
3. Localize o primeiro operando, *opnd1*, da string. Converta-o para posfixo e chame-o de *post1*
4. Encontre o segundo operando, *opnd2*, na string. Converta-o para posfixo e chame-o de *post2*
5. Concatene *post1*, *post2* e *po*

Eficiência da Recursividade

- Em geral, uma versão não-recursiva de um programa executará com mais eficiência do que uma versão recursiva
 - Existe um custo despendido para entrar e sair de um bloco
- Entretanto, por vezes uma solução recursiva é o método mais natural e lógico de solucionar um problema
 - Torres de Hanoi
 - Conversão prefixa para posfixa

Eficiência de Máquina x Programador

- Conflito entre a eficiência da máquina e a do programador
 - Custo da programação aumentando consideravelmente
 - Custo da computação diminuindo
 - Qual devemos privilegiar??
- Se um programa precisa ser executado com frequência, o investimento extra no tempo de programação será compensado
 - Mesmo nesses casos, é provável que seja melhor criar uma versão não-recursiva simulando e transformando a solução recursiva
 - Melhor do que criar uma solução não-recursiva a partir do enunciado do problema

Pilha

- Quando uma pilha não pode ser eliminada da versão não-recursiva de um programa
- E quando a versão recursiva não contém nenhum dos parâmetros adicionais ou variáveis locais, a versão recursiva pode ser tão ou mais veloz que a versão não-recursiva
 - Sob um compilador eficiente

Resumo

- As ideias e transformações que desenvolvemos ao apresentar a função fatorial e o problema das Torres de Hanoi podem ser aplicadas a problemas mais complexos
 - Cuja solução não-recursiva não esteja prontamente evidente

Exercício em Sala 1

Escreva uma função recursiva para adicionar os primeiros n termos da série abaixo:

$$1 + \frac{1}{2} - \frac{1}{3} + \frac{1}{4} - \frac{1}{5} \dots$$

Exercício em Sala 2

Escreva uma função recursiva que determina se um vetor é palíndromo ou não. Um palíndromo é uma palavra, frase ou qualquer outra sequência de unidades que tenha a propriedade de poder ser lida tanto da direita para a esquerda como da esquerda para a direita. Alguns exemplos são: “o galo ama o lago”, “Anotaram a data da maratona”, “E até o Papa poeta é”, 12321 e 1221.

Referências

- TENENBAUM, Aaron M; ANGENTEIN, Moshe; LANGSAM, Yedidiah. Estruturas de dados usando C. Sao Paulo, SP: Pearson, 1995. 884p.
- FEOFILOFF, Paulo. Algoritmos em linguagem C. Rio de Janeiro: Elsevier, 2009. 208 p.